Day 11 — Reverse a Number (Part 1)

Problem Statement

Ek integer `n` diya hai.

Uske digits ko reverse karke return karo.

Example 1

```
Input:
12345

Output:
54321
```

Example 2

```
Input:
9070

Output:
709
```

Leading zero automatically remove ho jate hain.

Example 3

```
Input:
-123

Output:
-321
```

Kitni Approaches Hoti Hain?

Is problem me actual DSA me mainly ye approaches use hoti hain.

Approach 1 (Mathematical / Modulo + Division) □□□□□

Ye **standard interview approach** hai.

Time : O(log10 N)

Space : O(1)

Almost har company isi approach ki expectation rakhti hai.

Approach 2 (String Conversion)

Useful hai.

Lekin DSA interviews me generally preferred nahi hoti.

Kabhi kabhi beginner understanding ke liye puch sakte hain.

Approach 3 (Overflow Safe Reverse)

Ye actually Approach 1 ka hi improved version hai.

LeetCode aur interviews me bahut important hai.

Jab question me bola ho

Reverse integer without overflow.

Tab ye use hoti hai.

Isliye hum sirf ye 3 approaches padhenge.

Koi unnecessary approach nahi hai.

Pehle Logic Samajhte Hain

Example

`1234`

Reverse banana hai

`4321`

Question hai...

Number ke andar se digits kaise nikale?

Answer:

Modulo `%`

Example

```
1234 % 10
```

```
=
```

```
4
```

Last digit mil gaya.

Ab us digit ko hatao.

```
1234 / 10
```

```
=
```

```
123
```

Again

```
123 % 10
```

```
=
```

```
3
```

Again

```
123 /10
```

```
=
```

```
12
```

Fir

```
12 %10 =2
```

```
12 /10 =1
```

```
1 %10 =1
```

```
1/10=0
```

Ab number khatam.

Observation

Har iteration me

```
Last digit nikalo
```

↓

```
Answer me add karo
```

↓

```
Last digit hata do
```

Bas.

Yehi pura logic hai.

Approach 1 (Mathematical Method)

Idea

Har baar

```
digit = n % 10
```

Reverse number me add karte jao.

Lekin add kaise?

Example

Current reverse

```
54
```

Ab digit mila

```
3
```

Answer hona chahiye

```
543
```

Direct

```
54+3
```

=

```
57
```

```
Wrong
```

To pehle

```
54
```

ko

```
540
```

banana padega.

Kaise?

```
54*10
```

```
=
```

```
540
```

Fir

```
540+3
```

```
=
```

```
543
```

Isliye formula

```
reverse = reverse * 10 + digit
```

Ye formula bahut important hai.

Interview me isi formula ka use hota hai.

Dry Logic

Suppose

```
n=1234
```

```
reverse=0
```

Iteration 1

```
digit=4
```

```
reverse
```

```
=
```

```
0*10+4
```

```
=
```

```
4
```

Iteration 2

```
digit=3
```

```
reverse
```

```
=
```

```
4*10+3
```

```
=
```

```
43
```

Iteration 3

```
digit=2
```

```
reverse
```

```
=
```

```
43*10+2
```

```
=
```

```
432
```

Iteration 4

```
digit=1
```

```
reverse
```

=

432*10+1

=

4321

Done.

Java Code

```
import java.util.Scanner;

public class ReverseNumber {

    public static int reverse(int n) {

        int reverse = 0;

        while (n != 0) {

            int digit = n % 10;

            reverse = reverse * 10 + digit;

            n = n / 10;
        }

        return reverse;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number : ");

        int n = sc.nextInt();

        System.out.println(reverse(n));
    }
}
```

Code Line by Line

```
int reverse = 0;
```

Reverse number store karega.

Initially

```
0
```

```
while(n!=0)
```

Jab tak digits bachi hain.

Example

```
1234

↓

123

↓

12

↓

1

↓

0

Stop
```

```
int digit=n%10;
```

Last digit nikalo.

Example

```
5678

↓

8
```

```
reverse=reverse*10+digit;
```

Sabse important line.

Example

```
reverse

=

43

digit

=

2
43*10+2

=

432
```

```
n=n/10;
```

Last digit hata do.

Example

```
987

↓

98
```

```
return reverse;
```

Final reverse return.

Complete Dry Run

Input

```
n=5829
```

Initially

```
reverse=0
```

Iteration 1

```
digit

=

5829%10

=
```

```
9
reverse

=

0*10+9

=

9
n

=

582
```

Iteration 2

```
digit

=

2
reverse

=

9*10+2

=

92
n

=

58
```

Iteration 3

```
digit

=

8
reverse

=

92*10+8

=

928
n
```

```
=

5
```

Iteration 4

```
digit

=

5
reverse

=

928*10+5

=

9285
n

=

0
```

Loop stop.

Answer

```
9285
```

Time Complexity

Har iteration me ek digit remove hoti hai.

Agar number me `d` digits hain, to loop `d` baar chalega.

Aur kisi integer `N` me digits ki count hoti hai:

```
d = log10(N) + 1
```

Isliye:

```
Time Complexity = O(log10 N)
```

Proof

Example:

```
N = 123456
```

Digits = 6

Loop = 6 times

`6 ≈ log10(123456)`

Isliye complexity `O(log N)` (base 10) hoti hai.

Space Complexity

Hum sirf 3 variables use kar rahe hain:

- `reverse`
- `digit`
- `n`

Input size badhne par extra memory nahi badhti.

```
Space Complexity = O(1)
```

Advantages

- Fastest practical solution.
- No extra memory.
- Interview standard approach.
- Competitive Programming me bhi isi ka use hota hai.

Disadvantages

- Agar reverse integer range se bahar chala jaye to overflow ho sakta hai.
- Overflow handling alag se karni padti hai (ye hum Part 2 me dekhenge).

Common Mistakes

1.

```
reverse = reverse + digit;
```

 Wrong

Har baar pehle `*10` karna zaruri hai.

2.

```
n % 2
```

 Wrong

Digit nikalne ke liye `%10` use hota hai.

3.

```
n = n - 10;
```

 Wrong

Last digit hatane ke liye:

```
n = n / 10;
```

4.

Loop:

```
while(n>0)
```

Ye positive numbers ke liye sahi hai, lekin negative numbers ke liye nahi chalega.

Edge Cases

Case 1

```
Input
```

```
0
```

Output

```
0
```

Case 2

```
1000
```

Output

```
1
```

Leading zero remove ho jate hain.

Case 3

```
10
```

Output

```
1
```

Case 4

```
7
```

Output

```
7
```

Interview Me Kaunsi Approach Likhen?

 Approach 1 (Mathematical / Modulo + Division)

Kyun?

- O(log N) time.
- O(1) space.
- Extra memory nahi lagti.
- Yehi standard DSA aur coding interview approach hai.

Part 2 me hum cover karenge:

1. **Negative Number Handling**
2. **Approach 2 (String Method)** – Kab use karni chahiye aur kab avoid.
3. **Approach 3 (Overflow Safe Reverse Integer)** – LeetCode aur interview ka sabse important variant.
4. Overflow check ka logic line-by-line aur dry run.

Day 11 — Reverse a Number (Part 2)

Ab hum **Approach 2 (String Method)** aur **Negative Numbers** ko cover karenge.

Note: Ye approach DSA interviews me primary approach nahi mani jati. Interview me pehle Mathematical Approach likho. String approach sirf tab use karo jab interviewer allow kare ya specifically bole.

Approach 2 — String Method

Idea

Number ko pehle **String** me convert kar do.

Fir String ko reverse karo.

Uske baad wapas integer bana do.

Example:

```
12345

↓

"12345"

↓

"54321"

↓

54321
```

Bas itna hi.

Step by Step

Input

```
1234
```

Step 1

```
String s = "1234"
```

Step 2

Reverse String

```
4321
```

Step 3

Convert back

```
4321
```

Done.

Java Code

```
import java.util.Scanner;

public class ReverseNumberString {

    public static int reverse(int n) {

        boolean negative = false;

        if (n < 0) {
            negative = true;
            n = -n;
        }

        String s = Integer.toString(n);

        StringBuilder sb = new StringBuilder(s);

        sb.reverse();

        int ans = Integer.parseInt(sb.toString());

        if (negative) {
            ans = -ans;
        }

        return ans;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number : ");

        int n = sc.nextInt();

        System.out.println(reverse(n));
```

```
    }
}
```

Line by Line Explanation

Line

```
boolean negative = false;
```

Ye check karega ki number negative hai ya nahi.

Line

```
if (n < 0)
```

Agar input

```
-123
```

to

```
negative = true
```

kar denge.

Fir

```
n = -n;
```

Matlab

```
-123
```

```
↓
```

```
123
```

Kyuki String reverse karte waqt minus sign ko reverse nahi karna.

Line

```
String s = Integer.toString(n);
```

Example

```
1234

↓

"1234"
```

Line

```
StringBuilder sb = new StringBuilder(s);
```

Ek mutable String ban gayi.

```
1234
```

Line

```
sb.reverse();
```

Ye built-in function hai.

```
1234

↓

4321
```

Line

```
Integer.parseInt(sb.toString());
"4321"

↓

4321
```

Line

```
if (negative)
```

Agar original number negative tha

```
4321

↓

-4321
```

Dry Run

Input

```
5829
```

Step 1

```
"5829"
```

Step 2

```
StringBuilder
↓
5829
```

Step 3

```
reverse()
↓
9285
```

Step 4

```
parseInt()
↓
9285
```

Answer

```
9285
```

Dry Run (Negative)

Input

```
-456
```

Negative?

```
Yes
```

Remove minus

```
456
```

Convert

```
"456"
```

Reverse

```
654
```

Convert

```
654
```

Minus wapas lagao

```
-654
```

Answer

```
-654
```

Time Complexity

Conversion

```
O(d)
```

Reverse

```
O(d)
```

Parsing

```
O(d)
```

Total

`O(d)`

Aur

`d = log10(N)`

Isliye

`Time Complexity = O(log N)`

Space Complexity

String ban rahi hai.

StringBuilder bhi ban raha hai.

Extra memory use ho rahi hai.

`Space Complexity = O(log N)`

Advantages

- Samajhne me easy.
- Code chhota hota hai.
- Beginners ke liye achha.

Disadvantages

- Extra memory lagti hai.
- Pure mathematical solution nahi hai.
- Coding interviews me preferred nahi hoti.
- Competitive Programming me bhi usually use nahi karte.

Common Mistakes

Mistake 1

Negative handle nahi karna.

```
-123
```

↓

```
321-
```

Ye invalid hai.

Mistake 2

Reverse ke baad parse karna bhool jana.

```
"321"
```

Ye String hai.

Integer nahi.

Mistake 3

Leading zeros ko lekar confusion.

```
1200
```

↓

```
0021
```

```
Integer.parseInt("0021")
```

Automatically

```
21
```

ban jayega.

Edge Cases

Input

```
0
```

Output

```
0
```

Input

```
100
```

Output

```
1
```

Input

```
-100
```

Output

```
-1
```

Input

```
9
```

Output

```
9
```

Mathematical vs String Approach

Feature	Mathematical	String
Time	O(log N)	O(log N)
Space	O(1)	O(log N)
Interview	□ Best	□ Rare
Competitive Programming	□ Yes	□ Avoid
Extra Memory	No	Yes
Recommended	□□□□□	□□

Interview Me Kaunsi Likho?

Mathematical Approach (Modulo + Division)

Kyun?

- O(1) Space
- Standard DSA solution
- Har interviewer isi ki expectation rakhta hai
- Logic building bhi isi se hoti hai

String approach sirf alternative knowledge ke liye ya specific condition me use kar sakte ho.

Part 3 me kya cover hoga?

Ab hum **sabse important interview variant** padhenge:

- Reverse Integer with Overflow
- Overflow hota kaise hai?
- Overflow detect kaise karte hain?
- LeetCode famous question
- Complete interview solution
- Line-by-line explanation
- Dry run
- Common interview traps

Ye Day 11 ka sabse important part hoga.

Day 11 – Reverse a Number (Part 3)

Approach 3 – Reverse Integer with Overflow (Interview Most Important)

□□□□□

Ye **LeetCode, Amazon, Microsoft, Google, Adobe, Oracle, Zoho, TCS Digital, Walmart** jaise interviews me bahut common variant hai.

Problem Statement

Ek integer `n` diya hai.

Uska reverse return karo.

Lekin...

Agar reverse integer Java ke `int` range ke bahar chala jaye to **0 return karna hai**.

Java int Range

```
Minimum = -2147483648

Maximum = 2147483647
```

Is range ke bahar value gayi to **Integer Overflow** ho jayega.

Example 1

```
Input

123

Output

321
```

Example 2

Input

-456

Output

-654

Example 3 (Overflow)

Input

1534236469

Reverse

9646324351

Ye `int` range se bahar hai.

Output

0

Pehle Overflow Samjho

Suppose

```
reverse = 214748364
```

Ab next digit hai

8

Formula

```
reverse = reverse * 10 + digit;
```

Banega

2147483648

Lekin Java int maximum

2147483647

hai.

Matlab overflow.

Sabse Badi Mistake

Bahut log likhte hain

```
reverse = reverse * 10 + digit;

if(reverse > Integer.MAX_VALUE)
```

 Wrong

Kyu?

Overflow **pehle hi ho chuka hai**.

Java me overflow ke baad value galat ban jati hai.

Uske baad check karne ka koi fayda nahi.

Correct Logic

Overflow hone se **PEHLE** check karo.

Yahi interview ka main logic hai.

Question

Hum check kaise kare?

Suppose

```
reverse = 214748364
digit = 8
```

Formula

```
reverse*10+digit
```

Overflow karega.

To multiplication se pehle hi socho.

Observation

Java ka maximum

```
2147483647
```

Agar divide karo

```
2147483647 / 10
```

```
=
```

```
214748364
```

Ye safe limit hai.

Matlab

Agar

```
reverse
```

```
>
```

```
214748364
```

to

```
reverse*10
```

already overflow karega.

Isliye

```
if(reverse > Integer.MAX_VALUE / 10)
```

Overflow.

Lekin Ek Aur Case Hai

Suppose

`reverse`

`=`

`214748364`

Ye equal hai.

Ab digit

`7`

To

`2147483647`

Safe.

Lekin digit

`8`

To

`2147483648`

Overflow.

Isliye second condition bhi chahiye.

Final Positive Overflow Condition

```
if(reverse > Integer.MAX_VALUE / 10)
```

OR

```
reverse == Integer.MAX_VALUE / 10
```

Aur

```
digit > 7
```

Kyunki

```
Integer.MAX_VALUE
```

=

```
2147483647
```

Last digit

```
7
```

hai.

Negative Side

Minimum Integer

```
-2147483648
```

Last digit

```
-8
```

hai.

Condition

```
if(reverse < Integer.MIN_VALUE / 10)
```

OR

```
reverse == Integer.MIN_VALUE / 10
```

Aur

```
digit < -8
```

Tab overflow.

Final Conditions

```
if (reverse > Integer.MAX_VALUE / 10 ||
    (reverse == Integer.MAX_VALUE / 10 && digit > 7))
{
    return 0;
}
if (reverse < Integer.MIN_VALUE / 10 ||
    (reverse == Integer.MIN_VALUE / 10 && digit < -8))
{
    return 0;
}
```

Ye conditions interview me yaad nahi, **samajhni** hain.

Java Code

```
import java.util.Scanner;

public class ReverseInteger {

    public static int reverse(int n) {

        int reverse = 0;

        while (n != 0) {

            int digit = n % 10;

            // Positive Overflow
            if (reverse > Integer.MAX_VALUE / 10 ||
                (reverse == Integer.MAX_VALUE / 10 && digit > 7)) {
                return 0;
            }

            // Negative Overflow
            if (reverse < Integer.MIN_VALUE / 10 ||
                (reverse == Integer.MIN_VALUE / 10 && digit < -8)) {
                return 0;
            }

            reverse = reverse * 10 + digit;

            n = n / 10;
        }

        return reverse;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number : ");

        int n = sc.nextInt();

        System.out.println(reverse(n));
    }
}
```

Line by Line Explanation

Line

```
int reverse = 0;
```

Reverse number store karega.

Line

```
while (n != 0)
```

Jab tak digits bachi hain tab tak loop chalega.

Line

```
int digit = n % 10;
```

Last digit nikalo.

Example

```
5829
```

```
↓
```

```
9
```

Positive Overflow Check

```
if (reverse > Integer.MAX_VALUE / 10)
```

Example

```
reverse = 300000000
```

Ab

```
300000000 *10
```

Definitely overflow.

Isliye turant

```
return 0;
```

Equal Case

```
reverse == Integer.MAX_VALUE / 10
```

Example

```
214748364
```

Ye abhi safe hai.

Ab digit

```
7
```

Safe.

Lekin

```
8
```

Overflow.

Isliye

```
digit > 7
```

check karte hain.

Negative Check

Bilkul same logic.

Bas

```
MIN_VALUE
```

use hota hai.

Aur last digit

```
-8
```

hoti hai.

Main Formula

```
reverse = reverse * 10 + digit;
```

Ab hum safe hain.

Kyunki overflow pehle hi check kar liya.

Remove Last Digit

```
n = n / 10;
```

Next iteration.

Dry Run (Normal)

Input

```
123
```

Iteration 1

```
digit=3
```

```
reverse=3
```

```
n=12
```

Iteration 2

```
digit=2
```

```
reverse=32
```

```
n=1
```

Iteration 3

```
digit=1
```

```
reverse=321
```

```
n=0
```

Answer

```
321
```

Dry Run (Overflow)

Input

```
1534236469
```

Kuch iterations ke baad

```
reverse
```

```
=
```

```
964632435
```

Next digit

```
1
```

Ab check

```
964632435
```

```
>
```

```
214748364
```

Yes.

Matlab

```
reverse*10
```

Overflow karega.

Isliye

```
return 0;
```

Ho jayega.

Program crash nahi karega.

Time Complexity

Har iteration me ek digit remove hoti hai.

Agar number me `d` digits hain to loop `d` baar chalega.

Aur `d = log10(N) + 1`.

Isliye:

```
Time Complexity = O(log N)
```

Space Complexity

Sirf 2–3 variables use ho rahe hain:

- `reverse`
- `digit`
- `n`

Input ke size ke saath extra memory nahi badhti.

```
Space Complexity = O(1)
```

Advantages

- □ Interview standard solution.
- □ Overflow ko safely handle karta hai.
- □ O(1) extra space.
- □ LeetCode "Reverse Integer" ka expected solution.

Disadvantages

- Overflow conditions pehli baar me thodi confusing lag sakti hain.
- `Integer.MAX_VALUE / 10` aur last digit (`7`, `-8`) ka logic samajhna zaruri hai.

Common Mistakes

Mistake 1

```
reverse = reverse * 10 + digit;

if (reverse > Integer.MAX_VALUE)
```

 Overflow pehle hi ho chuka.

Mistake 2

Sirf positive overflow check karna.

 Negative overflow bhi check karna zaruri hai.

Mistake 3

```
digit > 8
```

 Wrong

Correct:

```
digit > 7
```

Kyunki `Integer.MAX_VALUE = 2147483647`.

Mistake 4

Negative ke liye

```
digit < -7
```

 Wrong

Correct:

```
digit < -8
```

Kyunki `Integer.MIN_VALUE = -2147483648`.

Interview Questions

Q1. Overflow ko multiplication ke baad check kyu nahi kar sakte?

Answer: Kyunki multiplication ke time hi overflow ho chuka hota hai. Baad me value already corrupt ho jati hai.

Q2. `digit > 7` hi kyu?

Answer: `Integer.MAX_VALUE = 2147483647`, iska last digit `7` hai. Agar last digit `8` ya `9` hua to value maximum range cross kar degi.

Q3. `digit < -8` hi kyu?

Answer: `Integer.MIN_VALUE = -2147483648`, iska last digit `-8` hai.